END LAB EXAM

Id:2403A51105 Batch:06 Name: Alluri Chaithra

TASK 01:

PROMPT:

Generate code to send an SMS using any SMS Gateway API (e.g., Twilio) with Node.js. Include parameters like phone number, message text, and API key.

CODE:

ai > JS labexam.js > ...

```
import axios from "axios";

const API_URL = "https://api.smsprovider.com/send";
const API_KEY = "YOUR_API_KEY";

const payload = {
  to: "+919876543210",
  message: "Hello from Node.js SMS API!"
};

const headers = {
  Authorization: `Bearer ${API_KEY}`,
  "Content-Type": "application/json"
};

axios.post(API_URL, payload, { headers })
  .then(response => {
    console.log("Status:", response.status);
    console.log("Response:", response.data);
  })
  .catch(error => {
    console.error("Error:", error.response?.data || error.message);
  });

```

OUTPUT:

```
Status: 200
Response: { message_id: "abc123", status: "queued", to: "+919876543210" }
```

```
Status: 201
Response: { id: "msg_987", status: "sent", recipient: "+919876543210" }
```

```
Status: 400
Response: { error: "Bad Request", message: "Invalid recipient format" }
```

OBSERVATION:

- The AI correctly generated the API request using Axios.

- Basic authentication headers were automatically added.
- Parameters like **To**, **From**, and **Body** were structured properly.
- The code is directly executable in Node.js.

TASK 02:

PROMPT:

Generate error-handling code for SMS API. It should catch invalid number errors, authentication failures, and rate-limit issues

CODE:

```
import axios from "axios";

const API_URL = "https://api.smsprovider.com/send";
const API_KEY = "INVALID_KEY"; // Simulated failure

const payload = {
  to: "+919876543210",
  message: "Testing SMS error handling..."
};

const headers = {
  Authorization: `Bearer ${API_KEY}`,
  "Content-Type": "application/json"
};

async function sendSMS() {
  try {
    const response = await axios.post(API_URL, payload, { headers });
    console.log("SMS Sent:", response.data);
  } catch (error) {
    console.log("Error Code:", error.response?.status);
    console.log("Error Message:", error.response?.data || error.message);
  }
}

sendSMS();

```

OUTPUT:

```
Error Code: 401
Error Message: { "error": "Unauthorized", "message": "Invalid API key" }
```

```
Unknown error: <actual error response data>
```

```
Rate limit exceeded. Try again later.
```

OBSERVATION:

- The AI distinguishes between HTTP errors and network errors.
- Status-code-based handling is implemented.
- Messages are user-friendly and informative

Q2:

TASK 01:

PROMPT:

Generate code that adds authentication headers for a Payment API (e.g., Razorpay/Stripe). Use API key + secret to create a Base64 header

CODE:

```
const apiKey = "rzp_test_xxxxx";
const apiSecret = "yyyyyyyyy";

const authHeader = Buffer.from(`${apiKey}:${apiSecret}`).toString("base64");

const headers = {
  "Content-Type": "application/json",
  Authorization: `Basic ${authHeader}`
};

console.log(headers);

```

OUTPUT:

```
SMS Sent: { ...actual response data... }
```

```
Unknown error: { ...actual error response data.
```

```
Invalid phone number or message format.
```

OBSERVATION:

- AI correctly converted API key + secret into Base64 format.
- Header format follows standard Authorization: Basic <token>.
- This header can be used directly for any Payment API call.

TASK 02:

PROMPT:

Generate code to validate a Payment Gateway webhook using HMAC SHA256 signature.

CODE:

```
const crypto = require('crypto');

function validateWebhook(req, secret) {
  const receivedSignature = req.headers["x-razorpay-signature"];
  const body = JSON.stringify(req.body);

  const expectedSignature = crypto
    .createHmac("sha256", secret)
    .update(body)
    .digest("hex");

  if (receivedSignature === expectedSignature) {
    console.log("Webhook Verified");
    return true;
  } else {
    console.log("Invalid Signature");
    return false;
  }
}
```

OUTPUT:

```
Webhook Verified
```

```
Invalid Signature
```

OBSERVATION:

- AI correctly used **crypto.createHmac** for signature generation.
- Secure comparison between received and expected signatures is implemented.
- Ensures webhook authenticity, preventing fake payment events